

SOC Home Lab Environment:
Log Onboarding and Threat Detection

Utibe Anthony Akpan

Akpanutibe342@gmail.com

Executive Summary

This report documents the design and implementation of a self-hosted Security Operations Center (SOC) home lab, developed to gain hands-on experience with the tools and workflows used in real-world security monitoring. Manually monitoring individual systems for threats is both time-consuming and inefficient. This project addressed that challenge by building a centralized detection environment using Wazuh as the core SIEM platform to monitor endpoints and network activity from a single dashboard. The lab includes Windows and Ubuntu endpoints, network intrusion detection through Suricata, and threat intelligence enrichment using the VirusTotal API. Custom correlation rules were also developed and tested to detect suspicious activity. Overall, this project demonstrates practical experience in building, configuring, and operating a functional SOC detection pipeline from the ground up.

Scope & Objective:

This project involved the design and implementation of a self-hosted Security Operations Center (SOC) home lab to develop hands-on, practical experience with the tools and workflows used in real-world SOC environments. The objective was to build a functional detection environment—from architecture and deployment to active monitoring—rather than relying solely on theoretical study.

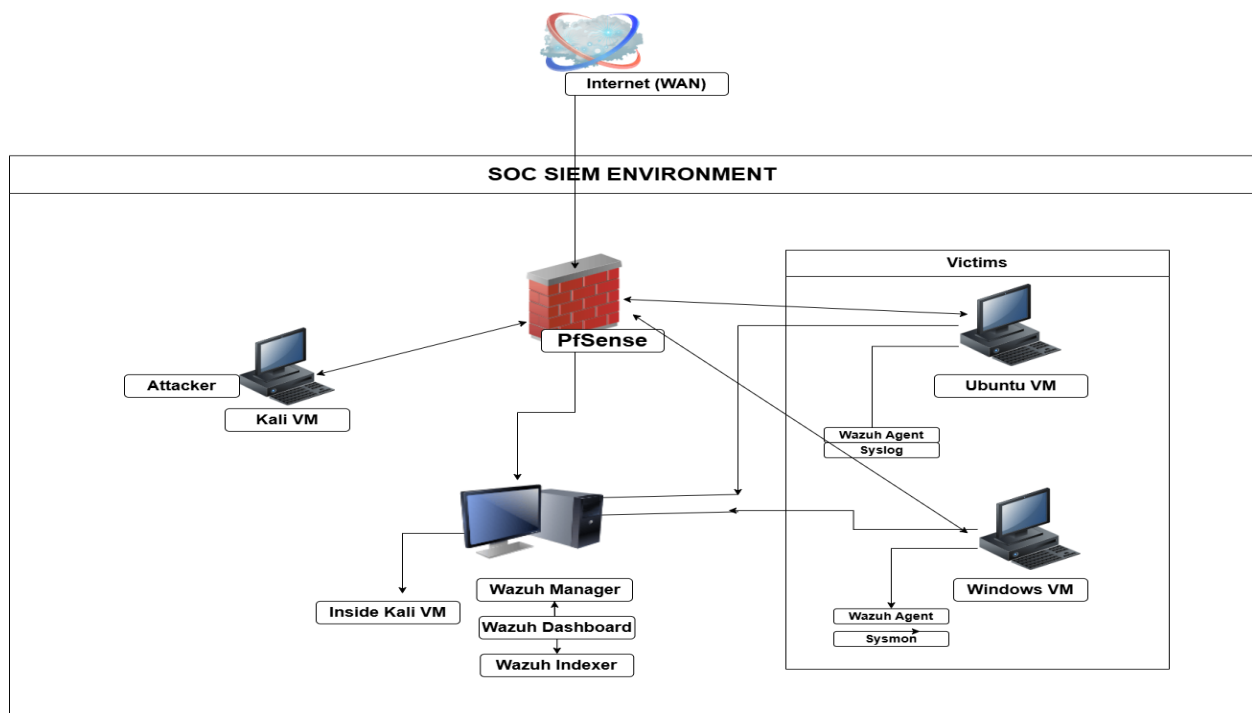
The lab environment consists of a Windows endpoint and an Ubuntu endpoint, both onboarded as monitored assets, with pfSense deployed as the perimeter firewall and router. Wazuh was configured as the central SIEM, serving as the manager, indexer, and dashboard. Suricata was integrated to provide network intrusion detection, while the VirusTotal API was connected to the Wazuh manager to enrich alerts with threat intelligence. On the Windows endpoint, PowerShell Script Block Logging was enabled to capture and forward detailed command-line activity to Wazuh for analysis.

Using this environment, custom correlation rules and dashboards were developed to detect and visualize suspicious activity. Each rule was tested against simulated events to validate detection accuracy. The overall goal was to demonstrate the ability to independently build, configure, and operate a working SOC detection pipeline, from raw log ingestion to actionable alerting.

Methodology:

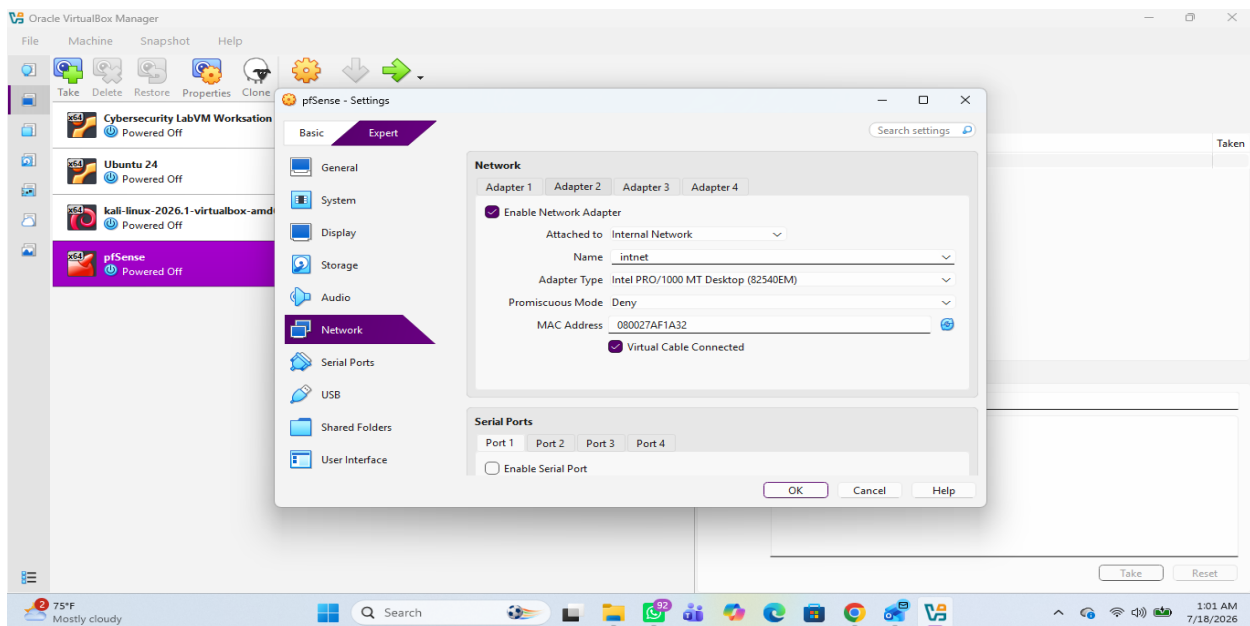
1. Architectural Planning

Before deploying any infrastructure, I designed an architectural diagram to map out the intended network topology. The diagram illustrated how the Wazuh manager, monitored endpoints, and the pfSense firewall would communicate with one another, with a simulated internet boundary positioned above the network. Planning the architecture first ensured that each component had a clear purpose and place within the environment before deployment began.



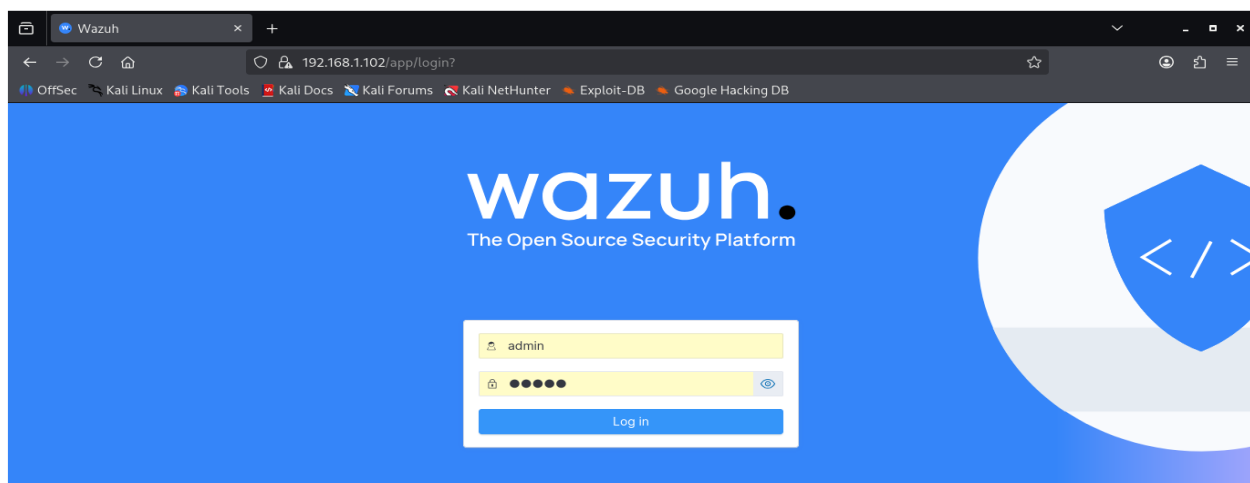
2. Environment Setup

With the design in place, I began building the lab environment using VirtualBox as the hypervisor. I downloaded and installed the required virtual machines, including Kali Linux, Ubuntu Server, Windows, and pfSense. These systems formed the foundation of the SOC lab and were configured to communicate within the virtual network.



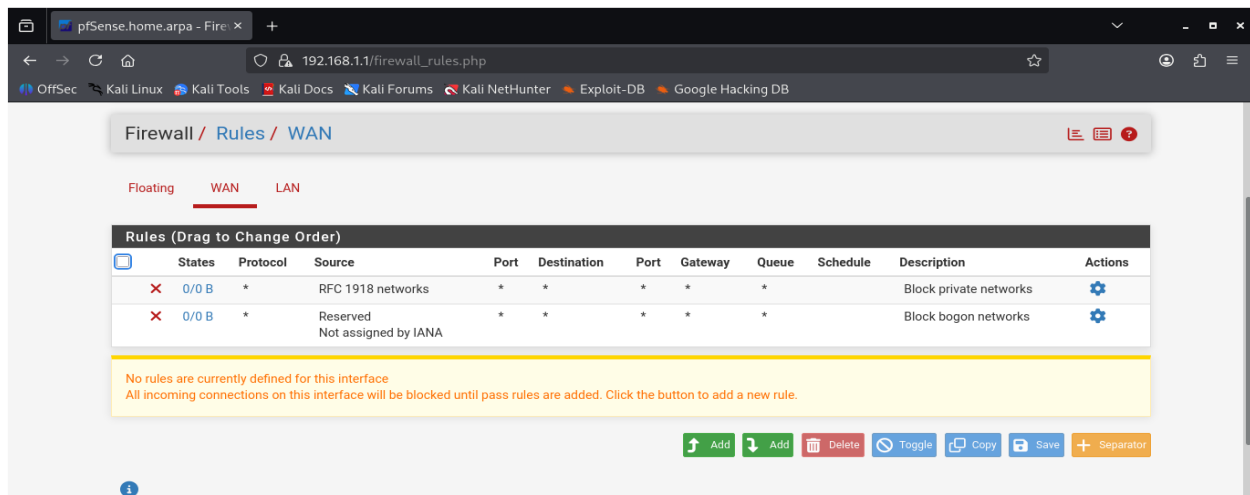
3. Wazuh Manager Deployment

Kali Linux was configured to host the Wazuh manager, indexer, and dashboard, forming the central management platform of the lab. This server was responsible for collecting, processing, correlating, and visualizing logs and security alerts from all monitored endpoints.



4. pfSense Firewall Deployment

pfSense was deployed as the firewall and router for the virtual environment, simulating a real network boundary between the internal lab network and the external internet. This provided network segmentation and traffic control, creating a more realistic enterprise environment.



5. Onboarding the Ubuntu Endpoint

To onboard the Ubuntu virtual machine as a monitored endpoint, I navigated to the Agents section of the Wazuh dashboard, selected Add Agent, entered a name for the endpoint, and used the generated installation command to deploy the Wazuh agent on Ubuntu. After installation, I confirmed that the agent successfully registered and appeared as Active in the dashboard.

To improve network visibility, I installed and integrated Suricata on the Ubuntu endpoint. This added network intrusion detection capabilities alongside the host-based monitoring already provided by the Wazuh agent.

```
smart@smart-VirtualBox: ~/Pictures
smart@smart-VirtualBox:~/Pictures$ sudo systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/usr/lib/systemd/system/wazuh-agent.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-07-30 04:54:06 WAT; 15s ago
     Process: 161201 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 29 (limit: 2261)
   Memory: 179.3M (peak: 225.2M)
      CPU: 11.701s
   CGroup: /system.slice/wazuh-agent.service
           └─161247 /var/ossec/bin/wazuh-execd
              161255 /var/ossec/bin/wazuh-agentd
              161269 /var/ossec/bin/wazuh-syscheckd
              161279 /var/ossec/bin/wazuh-logcollector
              161286 /var/ossec/bin/wazuh-modulesd

Jul 30 04:54:03 smart-VirtualBox systemd[1]: Starting wazuh-agent.service - Wazuh agent...
Jul 30 04:54:03 smart-VirtualBox env[161201]: Starting Wazuh v4.14.5...
Jul 30 04:54:03 smart-VirtualBox env[161201]: Started wazuh-execd...
Jul 30 04:54:04 smart-VirtualBox env[161201]: Started wazuh-agentd...
Jul 30 04:54:04 smart-VirtualBox env[161201]: Started wazuh-syscheckd...
Jul 30 04:54:04 smart-VirtualBox env[161201]: Started wazuh-logcollector...
Jul 30 04:54:04 smart-VirtualBox env[161201]: Started wazuh-modulesd...
Jul 30 04:54:06 smart-VirtualBox env[161201]: Completed.
Jul 30 04:54:06 smart-VirtualBox systemd[1]: Started wazuh-agent.service - Wazuh agent.
smart@smart-VirtualBox:~/Pictures$
```

Wazuh						
192.168.1.102/app/threat-hunting#/overview/7tab=general&tabView=events&agentId=001&a=(filters:[]),query:(la...						
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB						
Threat Hunting smart-VirtualBox Index pattern wazuh-alerts-* API default a						
Jul 29, 2026 @ 15:56:58.888	smart-VirtualBox	Suricata: Alert - SURICATA QUIC failed decrypt	3	86601		
Jul 29, 2026 @ 15:56:58.879	smart-VirtualBox	Suricata: Alert - SURICATA QUIC failed decrypt	3	86601		
Jul 29, 2026 @ 15:56:58.870	smart-VirtualBox	Suricata: Alert - SURICATA QUIC failed decrypt	3	86601		
Jul 29, 2026 @ 15:56:58.862	smart-VirtualBox	Suricata: Alert - SURICATA QUIC failed decrypt	3	86601		
Jul 29, 2026 @ 15:56:58.488	smart-VirtualBox	Suricata: Alert - SURICATA QUIC failed decrypt	3	86601		
Jul 29, 2026 @ 15:56:58.480	smart-VirtualBox	Suricata: Alert - SURICATA QUIC failed decrypt	3	86601		
Jul 29, 2026 @ 15:56:58.469	smart-VirtualBox	Suricata: Alert - SURICATA QUIC failed decrypt	3	86601		
Jul 29, 2026 @ 15:56:58.459	smart-VirtualBox	Suricata: Alert - SURICATA QUIC failed decrypt	3	86601		
Jul 29, 2026 @ 15:56:58.450	smart-VirtualBox	Suricata: Alert - SURICATA QUIC failed decrypt	3	86601		
Jul 29, 2026 @ 15:56:58.442	smart-VirtualBox	Suricata: Alert - SURICATA QUIC failed decrypt	3	86601		
Jul 29, 2026 @ 15:10:08.791	smart-VirtualBox	Suricata: Alert - SURICATA QUIC failed decrypt	3	86601		
Jul 29, 2026 @ 15:10:08.784	smart-VirtualBox	Suricata: Alert - SURICATA QUIC failed decrypt	3	86601		
Jul 29, 2026 @ 15:10:08.777	smart-VirtualBox	Suricata: Alert - SURICATA QUIC failed decrypt	3	86601		

6. Onboarding the Windows Endpoint

The same onboarding process was followed for the Windows virtual machine by adding it as a new agent through the Wazuh dashboard, assigning a name, and installing the Wazuh agent using the generated command. After confirming the agent was active, I installed Sysmon to improve the quality and depth of Windows event logging.

To enhance visibility into script execution, I also enabled PowerShell Script Block Logging. Both Sysmon and PowerShell logs were forwarded to the Wazuh manager for centralized monitoring and analysis.

The screenshot shows the Wazuh dashboard interface. At the top, there's a navigation bar with the Wazuh logo and a search bar. Below it, the 'Endpoints' section displays three donut charts representing different agent states: Pending (0), Never connected (0), and Active (2). The 'Agents (2)' section below shows a table of active agents.

ID	Name	IP address	Group(s)	Operating system	Cluster node	Version	Status	Actions
002	Windows	192.168.1.100	default	Microsoft Windows 11 Home 10.0.26200.8037	node01	v4.14.5	active	👁️ ⚙️
001	smart-VirtualBox	192.168.1.103	default	Ubuntu 24.04.4 LTS	node01	v4.14.5	active	👁️ ⚙️

The screenshot shows the 'Threat Hunting' section of the Wazuh dashboard. It displays a table of events for agent 002, filtered by 'Windows'. The table includes columns for timestamp, event type, description, and a score.

Timestamp	Event Type	Description	Score	Wazuh ID
Jul 23, 2026 @ 19:24:57.850	Windows	CIS Microsoft Windows 11 Enterprise Benchmark v3.0.0: Ensure 'Enforce password hist...	3	19009
Jul 23, 2026 @ 19:19:59.826	Windows	SCA summary: CIS Microsoft Windows 11 Enterprise Benchmark v3.0.0: Score less than...	9	19005
Jul 23, 2026 @ 19:19:44.997	Windows	CIS Microsoft Windows 11 Enterprise Benchmark v3.0.0: Ensure 'Maximum password ag...	3	19015
Jul 23, 2026 @ 19:19:19.663	Windows	Powershell script querying system environment variables	4	91816
Jul 23, 2026 @ 19:19:11.760	Windows	Powershell script querying system environment variables	4	91816
Jul 23, 2026 @ 19:19:02.922	Windows	Powershell script querying system environment variables	4	91816
Jul 23, 2026 @ 19:18:54.362	Windows	Powershell script querying system environment variables	4	91816
Jul 23, 2026 @ 19:18:42.481	Windows	Powershell script querying system environment variables	4	91816
Jul 23, 2026 @ 19:18:27.745	Windows	Powershell script querying system environment variables	4	91816
Jul 23, 2026 @ 19:17:54.340	Windows	Powershell script querying system environment variables	4	91816
Jul 23, 2026 @ 19:17:40.502	Windows	Powershell script querying system environment variables	4	91816
Jul 23, 2026 @ 19:17:14.087	Windows	Wazuh agent started.	3	503

7. Threat Intelligence Integration

To strengthen the lab's detection capability, I integrated the VirusTotal API with the Wazuh manager. This enable alerts containing suspicious files, hashes, and other indicators of compromise to be automatically enriched with VirusTotal threat intelligence, providing additional context for investigation and triage.

	agent.name	rule.description	rule.level	rule.id
Jul 28, 2026 @ 03:00:41.389	smart-VirtualBox	VirusTotal: Alert - /root/snap/snapd-desktop-integration/current - No positives found	3	87104
Jul 28, 2026 @ 02:43:27.778	smart-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jul 28, 2026 @ 02:43:19.874	smart-VirtualBox	VirusTotal: Alert - /etc/systemd/system/snapd.mounts.target.wants/snap-snapd\x2ddes...	3	87104
Jul 28, 2026 @ 02:43:10.088	smart-VirtualBox	VirusTotal: Alert - /etc/systemd/system/multi-user.target.wants/snap-snapd\x2ddesкто...	3	87104
Jul 28, 2026 @ 02:43:05.092	smart-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jul 28, 2026 @ 02:27:27.459	smart-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jul 28, 2026 @ 02:27:23.698	smart-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jul 28, 2026 @ 02:27:23.698	smart-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103
Jul 28, 2026 @ 02:27:23.698	smart-VirtualBox	VirusTotal: Alert - No records in VirusTotal database	3	87103

Findings

1. Dashboards

Dashboard 1 — Authentication Activity

This dashboard tracks authentication activity across the environment, displaying successful and failed login attempts, a breakdown of system events by host and account or event type, and the top accounts generating alerts.

Dashboard 2 — Endpoint Process Activity

This dashboard monitors process activity across all endpoints, highlighting the most frequently executed processes, parent-child process relationships, and a real-time feed of recent process creation events.

Dashboard 3 — Network Traffic Overview

This dashboard visualizes network traffic over time, the top source and destination IP addresses, and the distribution of allowed versus denied network actions.

Title	Type	Description	Last updated	Actions
☐ Authentication Activity Dashboard	Dashboard	Displays successful vs failed logins/ system events, breakdown by host and account/event type, and Top account/top alert events table	Jul 29, 2026 @ 01:22:53.314	
☐ Endpoint Process Activity Dashboard	Dashboard	Monitors endpoint process activity cross all systems, including most frequently executed processes, parent-child relationships and a real-time feed of recent process creation events	Jul 29, 2026 @ 00:07:36.425	
☐ Network Traffic Overview Dashboard	Dashboard	Monitors network traffic volume over time, top src/dest talkers, and allow vs deny action distributions	Jul 29, 2026 @ 00:46:16.210	

Rows per page: 20

rule.description: Descending

@timestamp: Descending	rule.description: Descend...	Count
Jul 23, 2026 @ 18:39:16.203	Host-based anomaly detection	3
Jul 23, 2026 @ 18:39:16.203	PAM: Login session closed.	1
Jul 23, 2026 @ 18:39:16.203	PAM: Login session opened.	1
Jul 23, 2026 @ 18:39:16.203	Successful sudo to root user	1

rule.groups: Descending	rule.description: Descend...	Count
sca	SCA summary: CIS Distribution	19
sca	CIS Distribution Independent Li	10
sca	CIS Distribution Independent Li	9
sca	CIS Microsoft Windows 11 Ente	6

2. Correlation Rule 1 — Brute Force Detection

This rule monitors authentication events and triggers when a single username experiences five or more failed login attempts within a 300-second (5-minute) window from the same source IP address, which is a strong indicator of a brute-force attack. It is mapped to the MITRE ATT&CK technique T1110 – Brute Force.

- Rule ID: 100002
- Level: 10
- Frequency/Timeframe: 5 hits within 300 seconds
- Logic: Same source IP address, five or more failed login attempts within five minutes, resulting in a high-severity alert.

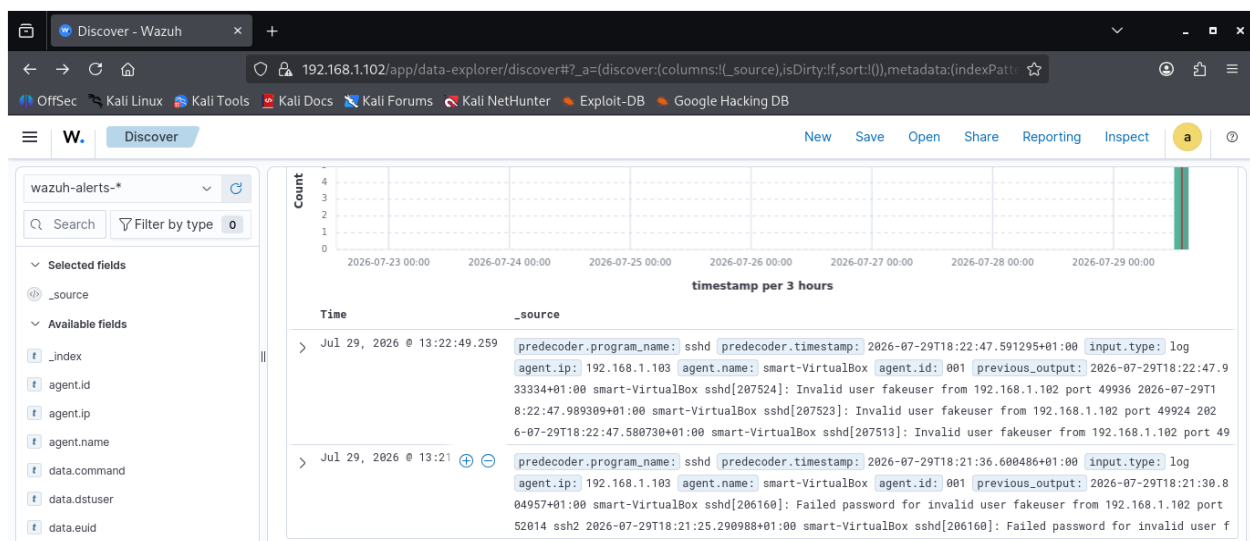
```
Session Actions Edit View Help
GNU nano 8.7.1 /var/ossec/etc/rules/local_rules.xml
! Local rules
-- Modify it at your will.
-- Copyright (C) 2015, Wazuh Inc.
-- Example
<group name="local,syslog,sshd,">
  <!-- Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2 -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>ssh: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>

  <!-- Rule 1: Failed Login Threshold (Brute Force) -->
  <rule id="100002" level="10" frequency="5" timeframe="300">
    <if_matched_sid>5760, 5712, 5716</if_matched_sid>
    <same_source_ip />
    <description>Possible SSH Brute Force Attack: 5 or more failed logins from the same IP within 5 minutes.</description>
    <mitre>
      <id>T1110</id>
    </mitre>
  </rule>

  <!-- Rule 2: Unusual Process Execution / Suspicious Directory Execution -->
  <rule id="100003" level="10" frequency="5" timeframe="300">
    <if_matched_sid>5760, 5712, 5716</if_matched_sid>
    <same_source_ip />
    <description>Possible Suspicious Process Execution: 5 or more failed logins from the same IP within 5 minutes.</description>
    <mitre>
      <id>T1059</id>
    </mitre>
  </rule>

  <!-- Rule 3: Suspicious Directory Execution -->
  <rule id="100004" level="10" frequency="5" timeframe="300">
    <if_matched_sid>5760, 5712, 5716</if_matched_sid>
    <same_source_ip />
    <description>Possible Suspicious Directory Execution: 5 or more failed logins from the same IP within 5 minutes.</description>
    <mitre>
      <id>T1059</id>
    </mitre>
  </rule>
</group>
```

Test Performed: I simulated a brute-force attack using a fake username and an incorrect password over SSH using Ubuntu as the victim and kali as the attacker, then used Hydra to automate repeated login attempts. The rule triggered successfully, generating an alert on the Wazuh dashboard.



3. Correlation Rule 2 — Suspicious Script/Binary Execution

This rule detects the execution of scripts or binaries from non-standard or suspicious locations, flagging activity that could indicate malicious execution. It is mapped to the MITRE ATT&CK technique T1059 – Command and Scripting Interpreter.

- Rule ID: 100003
- Level: 10
- Logic: Detects execution events that match predefined suspicious binary or script indicators.

```

GNU nano 8.7.1 /var/ossec/etc/rules/local_rules.xml

<!-- Rule 2: Unusual Process Execution / Suspicious Directory Execution -->
<rule id="100003" level="10">
  <if_sid>5402, 5501, 1002, 2901, 2902</if_sid>
  <regex>/tmp/|/var/tmp/</regex>
  <description>Suspicious execution detected: Binary or script executed from a temporary directory (/tmp/)</description>
  <mitre>
    <id>T1059</id>
  </mitre>
</rule>

</group>

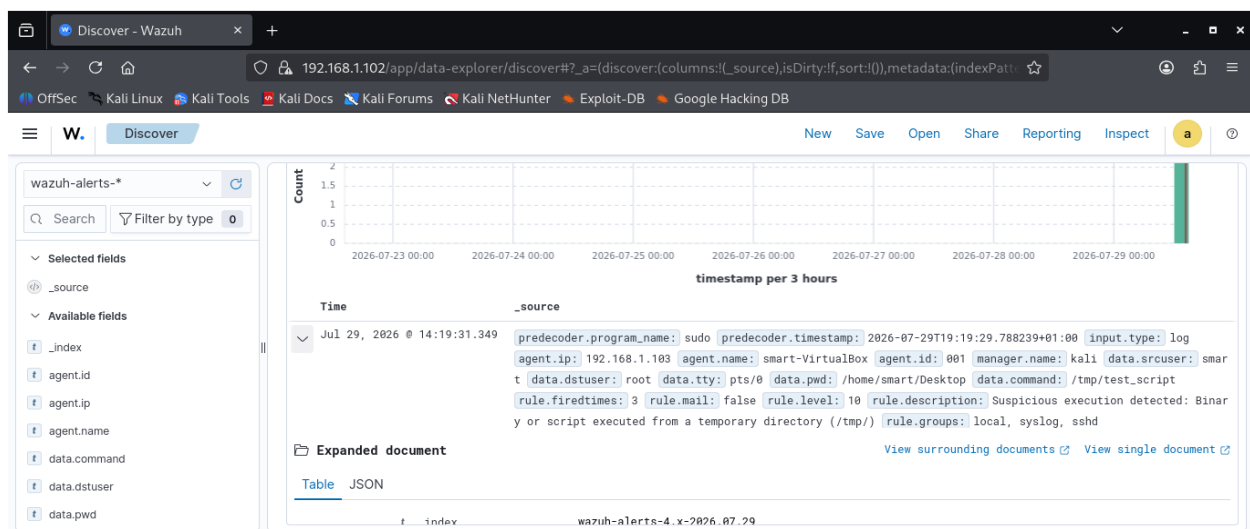
<!-- File modification -->
<rule id="100200" level="7">
  <if_sid>550</if_sid>
  <description>File modified in /root/</description>
</rule>

<rule id="100201" level="7">
  <if_sid>554</if_sid>
  <description>File added to /root/</description>
</rule>
</group>

<!-- File content evolution -->
<group name="pfsense,firewall">
  <rule id="100010" level="0">
    <program_name>filterlog</program_name>
  </rule>
</group>

```

Test Performed: I created a test script in the /tmp directory on the Ubuntu endpoint and executed it using sudo. The rule triggered successfully, and the alert appeared on the Wazuh dashboard.



Gaps / Challenges

Building this lab presented several real-world challenges, each of which strengthened my practical troubleshooting skills beyond the initial deployment process.

The first challenge was deploying pfSense. During the initial installation, the virtual machine failed to boot correctly and displayed error messages. A second attempt produced the same result. After investigating the issue, I discovered that the pfSense ISO image needed to be detached before restarting the virtual machine. Once the ISO was removed and the VM restarted, pfSense booted successfully. In hindsight, I would remove the ISO before the first restart attempt, as this would have avoided unnecessary troubleshooting time.

Deploying Wazuh also presented challenges, mainly due to network connectivity issues during installation. Instead of installing the manager, indexer, and dashboard components simultaneously, I installed and configured each component individually. This approach made it easier to isolate issues and troubleshoot them as they occurred.

In terms of visibility, the environment currently does not collect pfSense firewall logs. As a result, perimeter-level network traffic and firewall decisions are not yet visible within Wazuh. This remains a gap in the lab, and I am actively working on integrating pfSense logging into the SIEM pipeline to improve overall visibility.

Recommendations / Next Steps

Building on this initial deployment, several enhancements would further strengthen the environment's visibility, detection capability, and overall maturity:

- Integrate pfSense firewall logs into Wazuh to achieve complete network visibility, closing the current gap in perimeter-level monitoring.
- Develop additional custom detection rules mapped to the MITRE ATT&CK framework, expanding beyond the current two rules to cover a wider range of tactics and techniques associated with real-world threats.

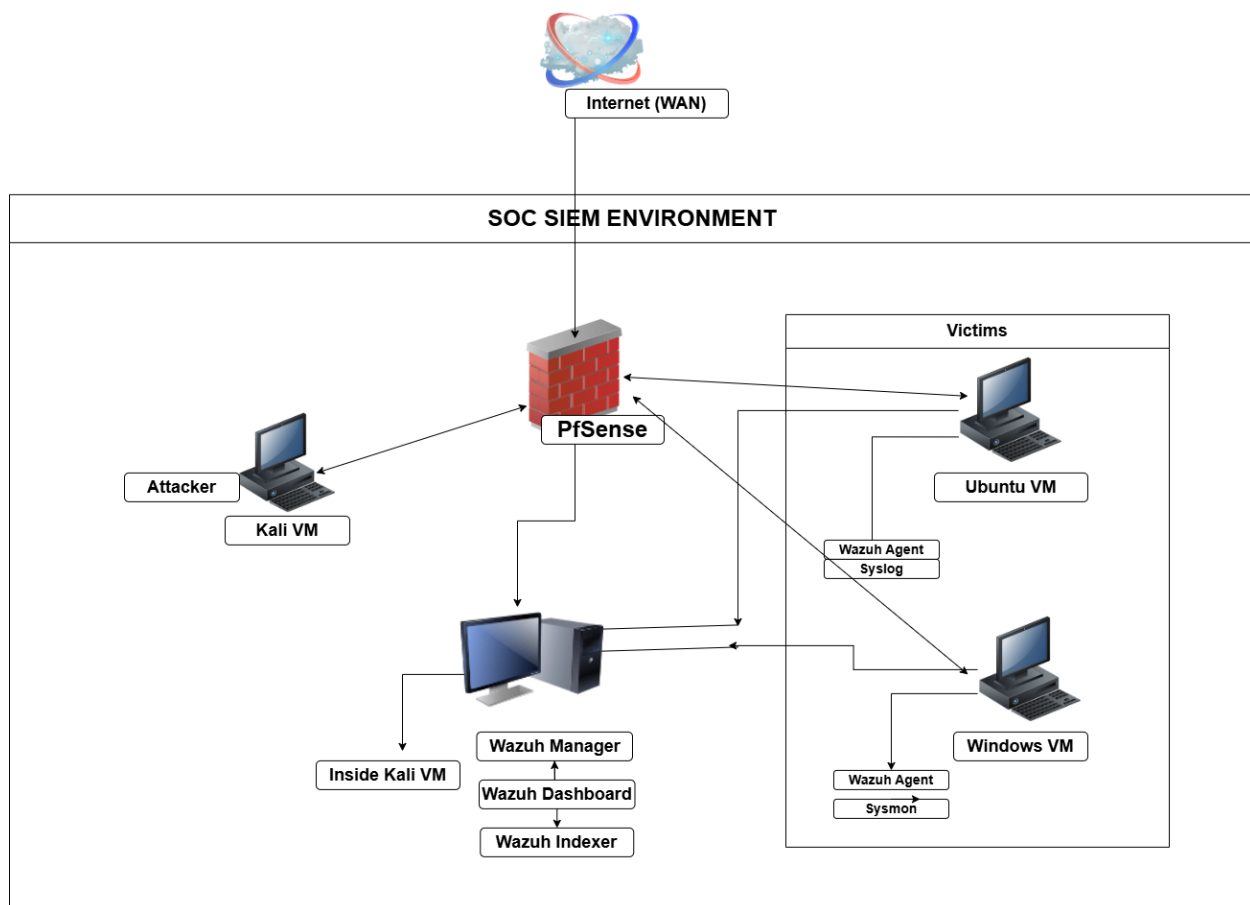
- Expand threat intelligence integration by incorporating additional threat intelligence sources alongside VirusTotal, enriching alerts with broader context and a wider range of indicators of compromise.

- Simulate additional attack scenarios, including ransomware, phishing, privilege escalation, and lateral movement, to validate detection rules against a broader threat landscape while continuing to strengthen investigation and correlation skills.

Appendix

A – SOC Lab Architecture Diagram

A diagram illustrating the overall architecture of the SOC home lab, including the Wazuh manager, monitored endpoints, pfSense firewall, and network layout.



B – Custom Wazuh Correlation Rule 1 (Brute Force Detection)

The XML configuration for the custom Wazuh rule developed to detect brute-force login attempts.

```
<!-- Rule 1: Failed Login Threshold (Brute Force) -->

<rule id="100002" level="10" frequency="5" timeframe="300">

  <if_matched_sid>5760, 5712, 5716</if_matched_sid>

  <same_source_ip />

  <description>Possible SSH Brute Force Attack: 5 or more failed logins from the same IP within
5 minutes.</description>

  <mitre>

    <id>T1110</id>

  </mitre>

</rule>
```

C – Custom Wazuh Correlation Rule 2 (Suspicious Script/Binary Execution)

The XML configuration for the custom Wazuh rule developed to detect suspicious script or binary execution.

```
<!-- Rule 2: Unusual Process Execution / Suspicious Directory Execution -->

<rule id="100003" level="10">

  <if_sid>5402, 5501, 1002, 2901, 2902</if_sid>

  <regex>/tmp/|/var/tmp/</regex>

  <description>Suspicious execution detected: Binary or script executed from a temporary
directory (/tmp/)</description>

  <mitre>

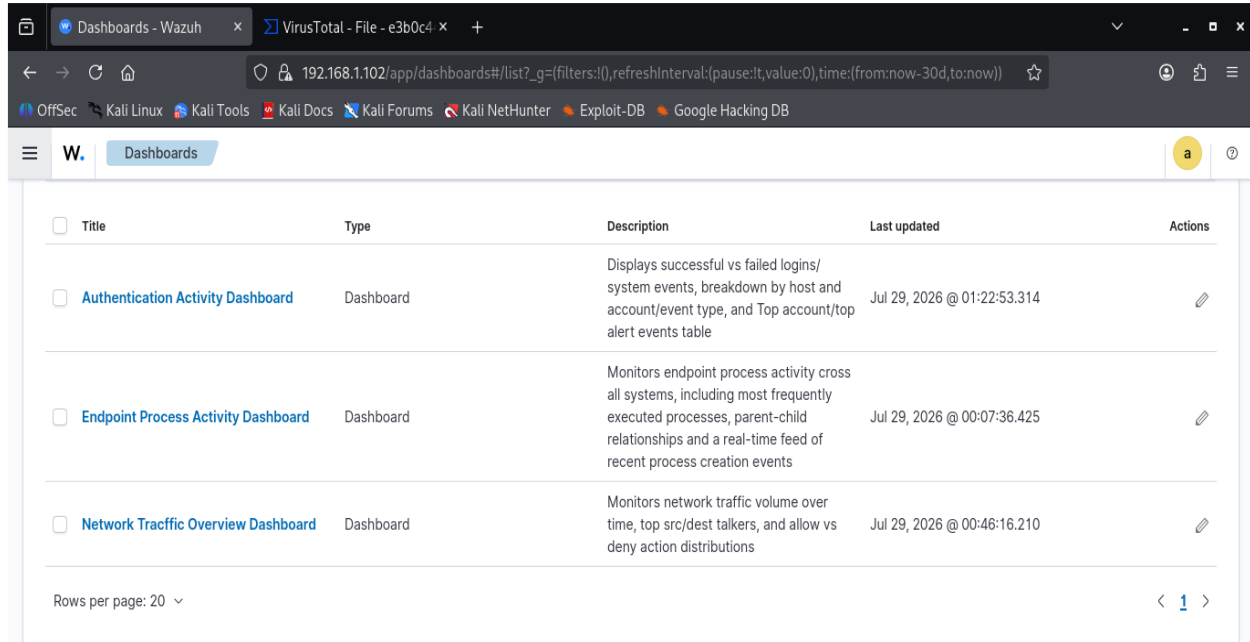
    <id>T1059</id>

  </mitre>

</rule>
```

D – Dashboard Screenshots

Additional dashboard screenshots showing the monitoring and visualization capabilities implemented in the SOC lab.

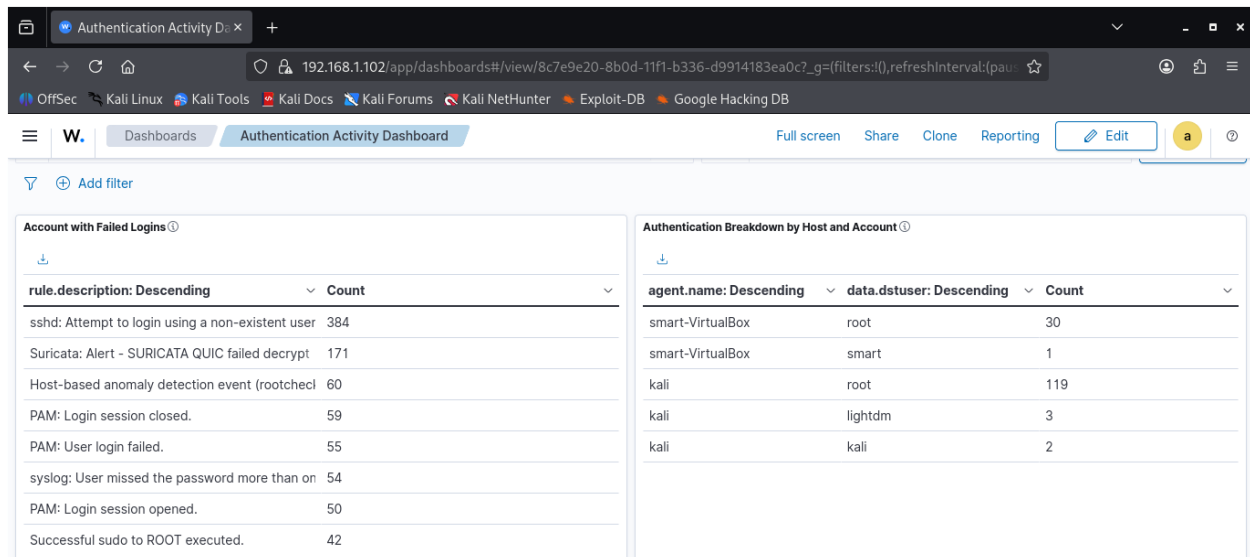


The screenshot shows the Wazuh Dashboards interface in a web browser. The browser tabs include 'Dashboards - Wazuh' and 'VirusTotal - File - e3b0c4'. The address bar shows the URL '192.168.1.102/app/dashboards#/list?_g=(filters:!(),refreshInterval:(pause:!t,value:0),time:(from:now-30d,to:now))'. The interface features a sidebar with a 'W.' logo and a 'Dashboards' tab. The main content area displays a table of available dashboards with columns for Title, Type, Description, Last updated, and Actions.

Title	Type	Description	Last updated	Actions
☐ Authentication Activity Dashboard	Dashboard	Displays successful vs failed logins/ system events, breakdown by host and account/event type, and Top account/top alert events table	Jul 29, 2026 @ 01:22:53.314	
☐ Endpoint Process Activity Dashboard	Dashboard	Monitors endpoint process activity cross all systems, including most frequently executed processes, parent-child relationships and a real-time feed of recent process creation events	Jul 29, 2026 @ 00:07:36.425	
☐ Network Traffic Overview Dashboard	Dashboard	Monitors network traffic volume over time, top src/dest talkers, and allow vs deny action distributions	Jul 29, 2026 @ 00:46:16.210	

Rows per page: 20

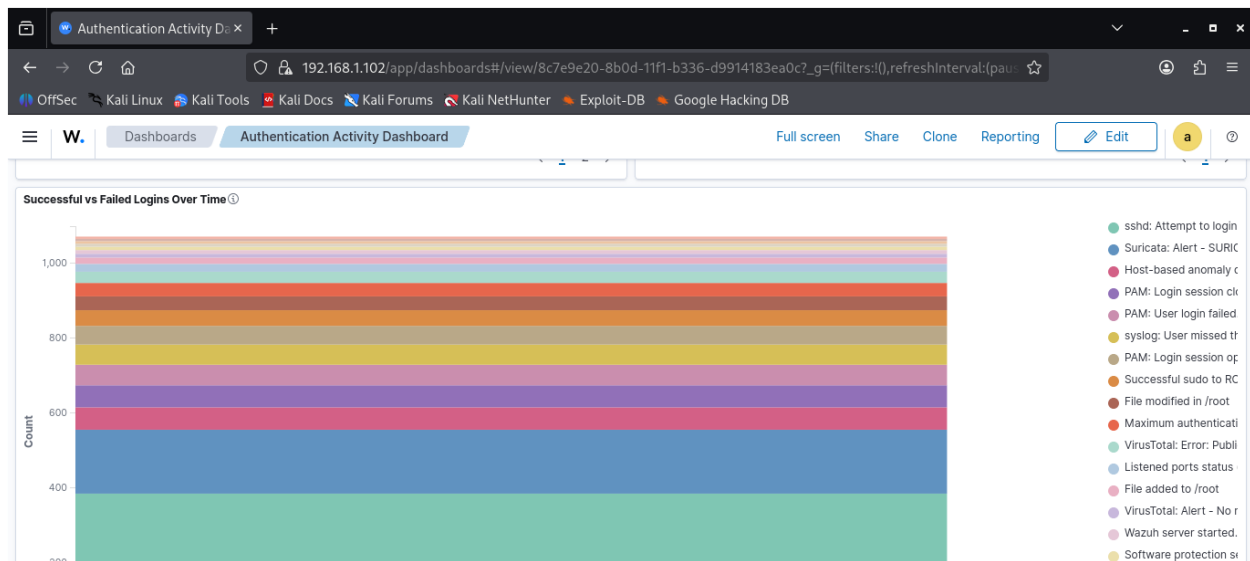
Authentication Activity Dashboard



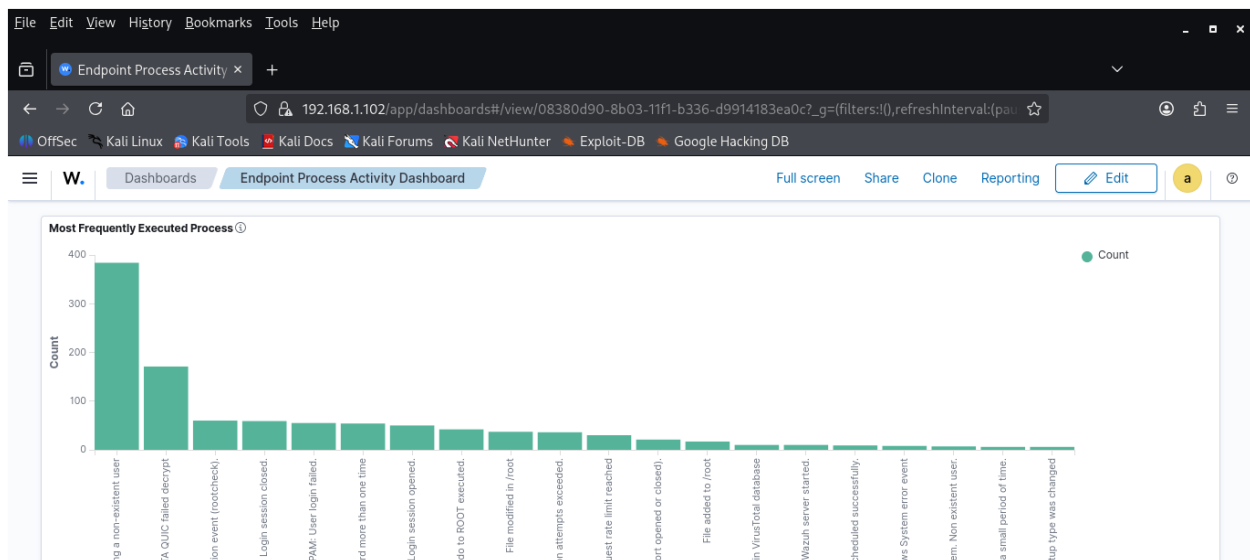
The screenshot shows the 'Authentication Activity Dashboard' in the Wazuh interface. The browser tabs include 'Authentication Activity D...' and the address bar shows the URL '192.168.1.102/app/dashboards#/view/8c7e9e20-8b0d-11f1-b336-d9914183ea0c?_g=(filters:!(),refreshInterval:(pau:'. The interface features a sidebar with a 'W.' logo and a 'Dashboards' tab. The main content area displays two data tables: 'Account with Failed Logins' and 'Authentication Breakdown by Host and Account'.

rule.description: Descending	Count
sshd: Attempt to login using a non-existent user	384
Suricata: Alert - SURICATA QUIC failed decrypt	171
Host-based anomaly detection event (rootcheck)	60
PAM: Login session closed.	59
PAM: User login failed.	55
syslog: User missed the password more than on	54
PAM: Login session opened.	50
Successful sudo to ROOT executed.	42

agent.name: Descending	data.dstuser: Descending	Count
smart-VirtualBox	root	30
smart-VirtualBox	smart	1
kali	root	119
kali	lightdm	3
kali	kali	2



Endpoint Process Activity Dashboard



Endpoint Process Activity

192.168.1.102/app/dashboards#/view/08380d90-8b03-11f1-b336-d9914183ea0c?_g=(filters:[]),refreshInterval:(pa

OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB

W.

Dashboards

Endpoint Process Activity Dashboard

Full screen Share Clone Reporting Edit

a

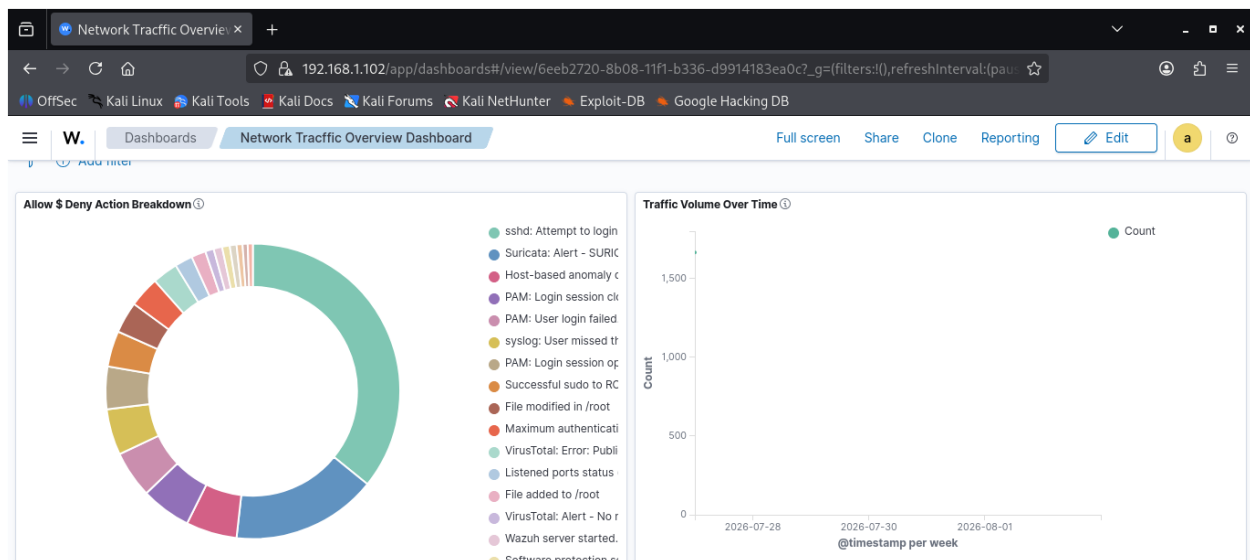
Process Creation Events

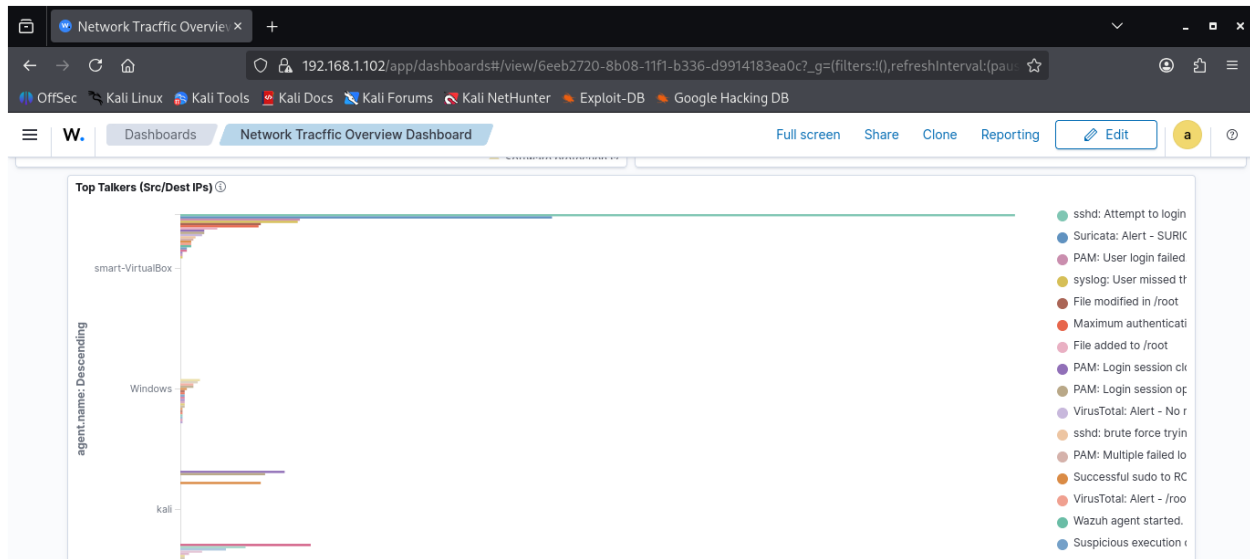
@timestamp: Descending	rule.description: Descend...	Count
Jul 29, 2026 @ 09:52:52.950	PAM: Login session closed.	2
Jul 29, 2026 @ 09:52:52.950	PAM: Login session opened.	2
Jul 29, 2026 @ 23:46:32.552	Host-based anomaly detection	3
Jul 29, 2026 @ 23:46:32.552	Listened ports status (netstat)	1
Jul 29, 2026 @ 10:39:30.479	PAM: Login session closed.	1
Jul 29, 2026 @ 10:39:30.479	PAM: Login session opened.	1
Jul 29, 2026 @ 10:39:30.479	Successful sudo to ROOT execi	1
Jul 29, 2026 @ 13:02:39.824	syslog: User missed the passw	2
Jul 29, 2026 @ 13:02:39.824	sshd: Attempt to login using a r	1

Parent-Child Process Relationships

rule.groups: Descending	rule.description: Descend...	Count
syslog	sshd: Attempt to login using a r	384
syslog	PAM: Login session closed.	59
syslog	PAM: User login failed.	55
syslog	syslog: User missed the passw	54
syslog	PAM: Login session opened.	50
syslog	Successful sudo to ROOT execi	42
syslog	Maximum authentication attem	36
syslog	sshd: brute force trying to get	7
syslog	PAM: Multiple failed logins in a	6

Network Traffic Overview Dashboard





Conclusion

This project successfully demonstrated the ability to design, deploy, and operate a functional SOC environment, from initial network architecture through active threat detection. While opportunities for improvement remain, particularly in integrating pfSense firewall logs for complete network visibility, the lab provided valuable hands-on experience in endpoint onboarding, dashboard creation, threat intelligence integration, and custom correlation rule development. It also established a solid foundation for expanding the environment with more advanced detection scenarios and continued SOC skill development.